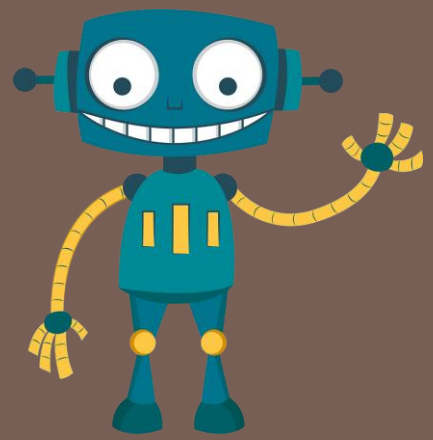
파이썬익스프레스





12장 상속



학습 목표

- 부모 클래스를 상속받아서 자식 클래스를 정의할 수 있다
- 부모 클래스의 메소드를 자식 클래스에서 재정의할 수 있다.
- Object 클래스를 이해할 수 있다.
- 메소드 오버라이딩을 이해하고 사용할 수 있다.
- 클래스 간의 관계를 파악할 수 있다.





이번 장에서 만들 프로그램

(1) 상속을 이용하여서 각 클래스에 중복된 정보를 부모 클래스로 모아보자. 구체적인 예로 Car 클래스와 ElectricCar 클래스를 작성해보자.

(2) 상속을 사용할 때, 자식 클래스와 부모 클래스의 생성자가 호출되는 순서를 살펴보자.

(3) 부모 클래스의 함수를 오버라이딩(재정의)하여 자식 클래스의 기능을 강력하게 하는 기법을 살펴보자.



객체지향의 3가지 핵심요소

- 캡슐화 : 데이터와 연산을 하나의 덩어리로 묶어서 다룸
  - 정보은닉을 위한 가시성 : public (공개), protected (보호), private(숨김)
    - 다른 클래스에서 이 클래스의 데이터와 연산을 접근하거나 접근불가로 만듬
    - Public: 모든 클래스에서 접근가능, protected: 자식만 접근가능, private : 접근 불가
- 상속 : 부모클래스를 자식클래스가 그대로 물려받아 추가적인 코드를 작성할 수 있음
  - 다만, 부모의 private 한 것은 상속은 받으나 접근 불가능
    - Protected는 외부 접근 불가능하나 자식이 물려받아 사용가능
- 다형성 : 함수 이름이 같더라도 문맥(혹은 클래스)에 따라 다른 함수가 수행됨
  - A.do(), B.do() 처럼 do()함수 이름은 같으나 클래스가 다르면 다른 함수
  - B가 A를 상속하고 재정의 할 경우 동적 바인딩이 가능



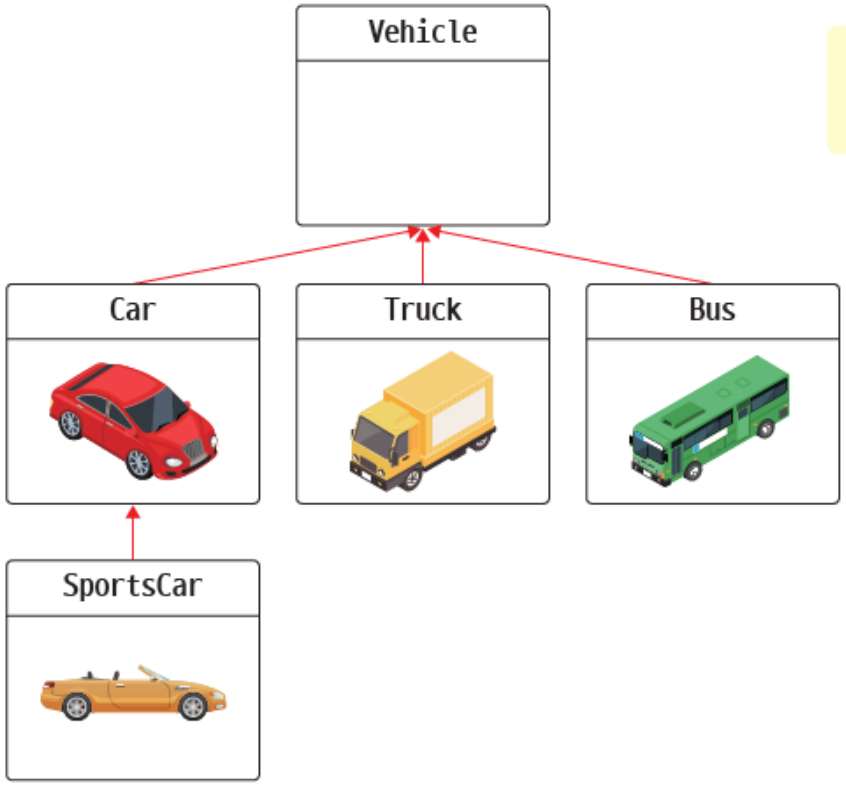
상속
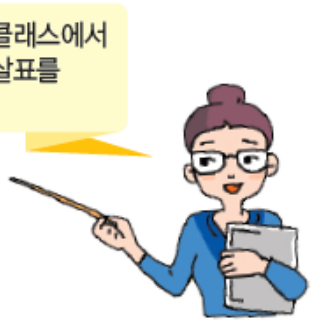
- 상속(inheritance)은 기존에 존 재하는 클래스로부터 코드와 데이터를 이어받고 자신이 필요한 기능을 추가하는 기 법이다.





상속의 예


Vehicle
Car
Truck
Bus
SportsCar


상속에서는 자식 클래스에서
부모 클래스로 화살표를
그립니다.



상속과 is-a 관계

- 객체 지향 프로그래밍에서는 상속이 클래스 간의 “is-a” 관계를 생성하는데 사용된다.

  - 푸들은 강아지이다.
  - 자동차는 차량이다.
  - 꽃은 식물이다.
  - 사각형은 모양이다.



상속의 예

부모 클래스	자식 클래스
Animal(동물)	Lion(사자), Dog(개), Cat(고양이)
Bike(자전거)	MountainBike(산악자전거), RoadBike, TandemBike
Vehicle(탈것)	Car(자동차), Bus(버스), Truck(트럭), Boat(보트), Motocycle(오토바이), Bicycle(자전거)
Student(학생)	GraduateStudent(대학원생), UnderGraduate(학부생)
Person(사람)	Student(학생), Employee(직원)
Shape(도형)	Rectangle(사각형), Triangle(삼각형), Circle(원)



상속 구현하기

Syntax: 상속 정의

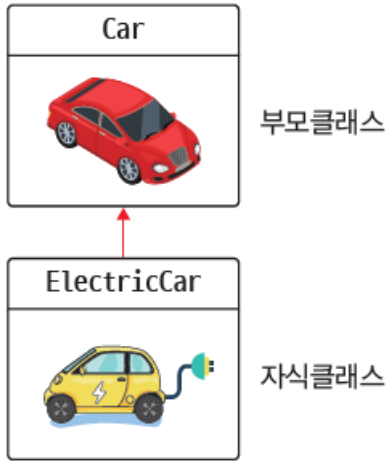
자식 클래스 또는 서브 클래스라고 한다.

Syntax

```
class 자식클래스(부모클래스) :
    def 메소드1(self, ...) :
        ...
    def 메소드2(self, ...) :
        ...
```

부모 클래스 또는 슈퍼 클래스라고 한다.

```
class ElectricCar(Car) :
    def setBatterySize(self, size):
        ...
    def getBatterySize(self):
        ...
```





예 제

```
# 일반적인 자동차를 나타내는 클래스이다.
class Car:
        def __init__(self, maker, model, color,  price):
                self.maker = maker                      # 메이커
                self.model = model              # 모델
                self.color = color              # 자동차의 색상
                self.price = price              # 자동차의 가격

        def setMake(self, maker):       # 설정자 메소드
                self.maker = maker

        def getMake(self):              # 접근자 메소드
                return self.maker

        # 차량에 대한 정보를 문자열로 요약하여서 반환한다.
        def getDesc(self):
                return "차량 =("+str(self.maker)+","+\
                        str(self.model)+","+\
                        str(self.color)+","+\
                        str(self.price)+")"
```

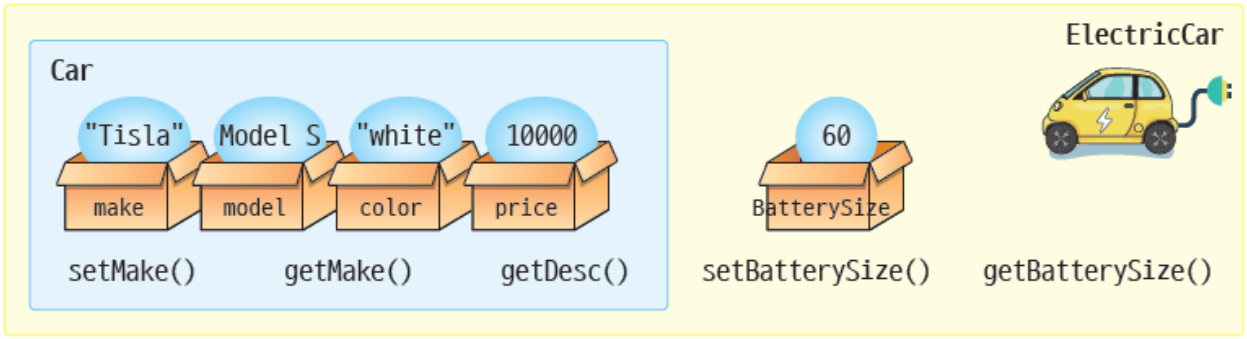


예 제

```
class ElectricCar(Car) :                                    # ①
        def __init__(self, make, model, color, price, batterySize):
                super().__init__(make, model, color, price)        # ②
                self.batterySize=batterySize                       # ③

        def setBatterySize(self,  batterySize):     # 설정자 메소드
                self.batterySize=batterySize

        def getBtterySize(self):                                   # 접근자 메소드
                return self.batterySize
```





예 제

```
def main():                                                    # main() 함수 정의

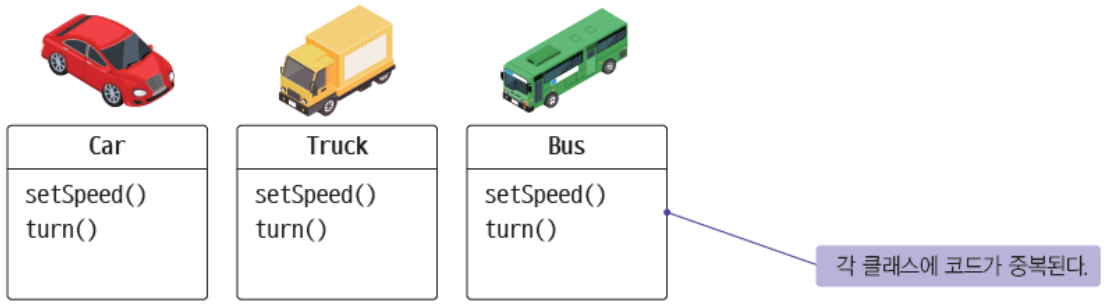
        myCar = ElectricCar("Tisla", "Model S", "white", 10000, 0)     #
        myCar.setMake("Tesla")                                 # 설정자 메소드 호출
        myCar.setBatterSize(60)                                # 설정자 메소드 호출
        print(myCar.getDesc())          # 전기차 객체를 문자열로 출력

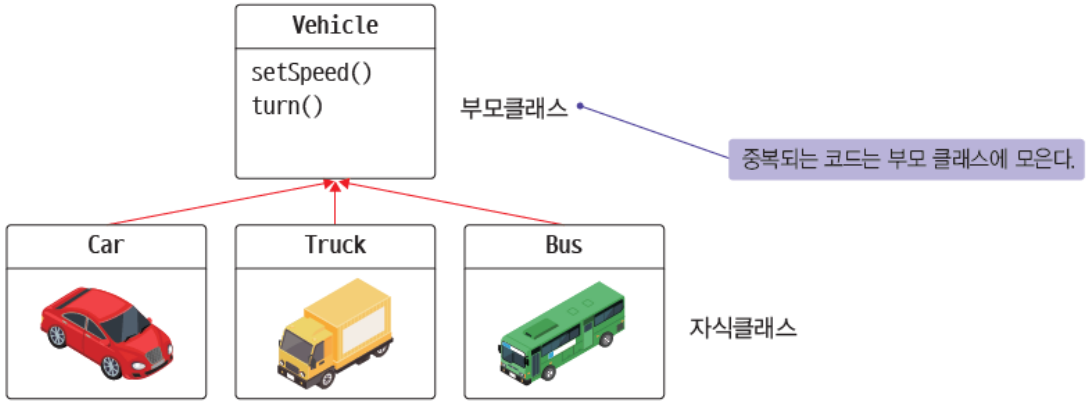
main()
```

```
차량 =(Tesla,Model S,white,10000)
```



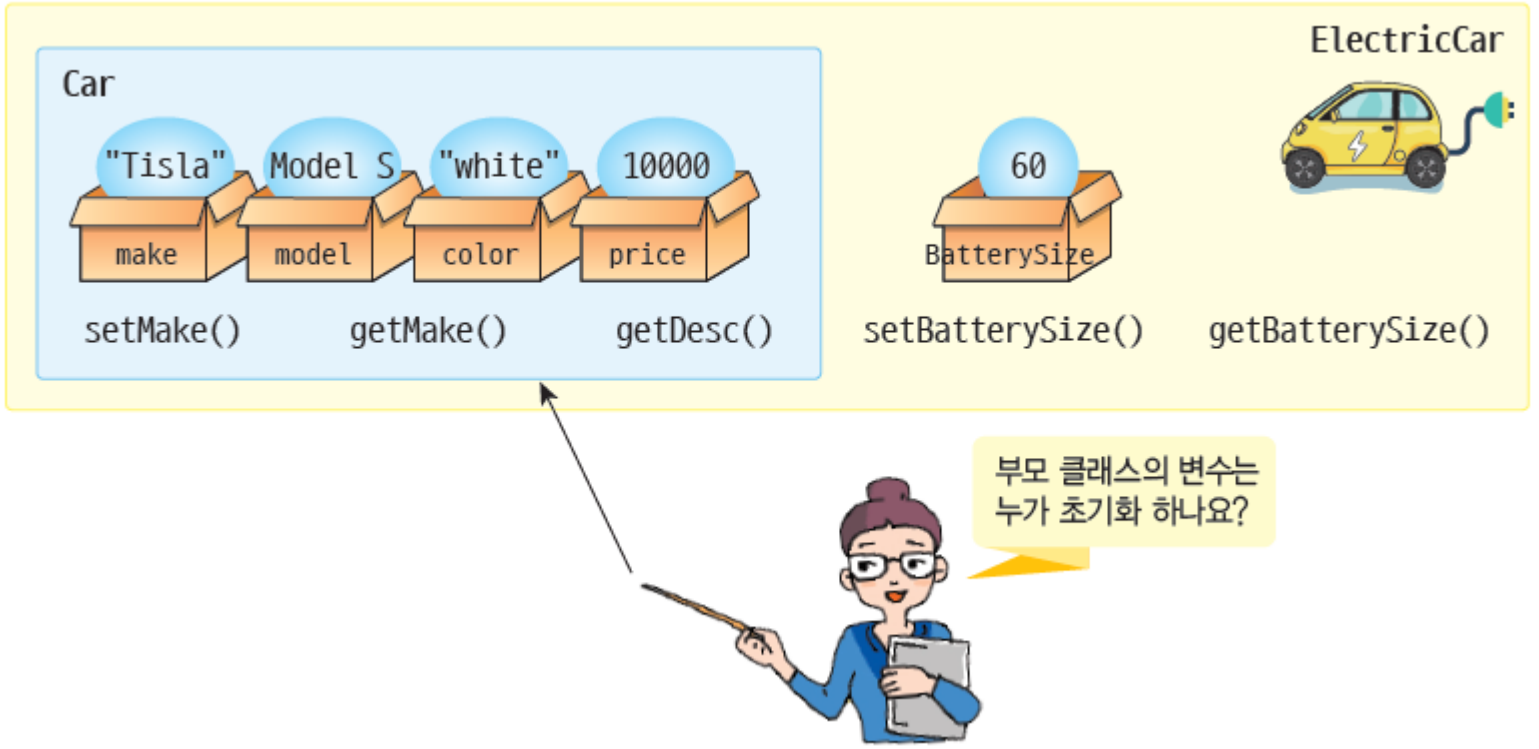
왜 상속을 사용하는가?


Car
setSpeed()
turn()
Truck
setSpeed()
turn()
Bus
setSpeed()
turn()
각 클래스에 코드가 중복된다.


Vehicle
setSpeed()
turn()
부모클래스
중복되는 코드는 부모 클래스에 모은다.
Car
Truck
Bus
자식클래스



부모 클래스의 생성자 호출





부모클래스 생성자를 호출하지 않으면 오류

- 부모클래스: Super-class 라고도 함 – super()는 자식클래스에서 부모클래스 참조 함수
- 자식클래스: Sub-class 라고도 함
- 자식클래스는 부모클래스의 모든 것을 상속받는다 : 속성, 연산 모든 것
  - 다만, 부모클래스의 private 속성을 가진 항목은 상속은 하되 접근할 수 없다.
  - 하지만, 부모클래스의 protected 속성을 가진 항목은 접근할 수 있다.
  - 물론, public 한 항목은 모든 클래스가 접근할 수 있다.

```
class ElectricCar(Car) :
        def __init__(self, make, model, color, price, batterySize):
                super().__init__(make, model, color, price)
                self.batterySize=batterySize
```



부모클래스 생성자를 호출하지 않으면 오류

```
class Animal:
    def __init__(self, age=0):
        self.age=age

    def eat(self):
        print("동물이 먹고 있습니다. ")


# 부모 클래스의 생성자를 호출하지 않았다!
class Dog(Animal):
    def __init__(self, age=0, name=""):
        self.name=name

# 부모 클래스의 생성자가 호출되지 않아서 age 변수가 생성되지 않았다.

d = Dog();
print(d.age)
```



type()과 isinstance() 함수

```
...
x = Animal();
y = Dog();
print(type(x))
print(type(y))
```

```
<class '__main__.Animal'>
<class '__main__.Dog'>
```



type()과 isinstance() 함수

```
...
x = Animal()
y = Dog()
print(isinstance(x, Animal), isinstance(y, Animal))
```

```
True True
```



private 멤버 부모 클래스

```
class Parent(object):
    def __init__(self):
        self.__money = 100 # private가 되어서 자식 클래스에서 사용할 수 없다.

class Child(Parent):
    def __init__(self):
        super().__init__()

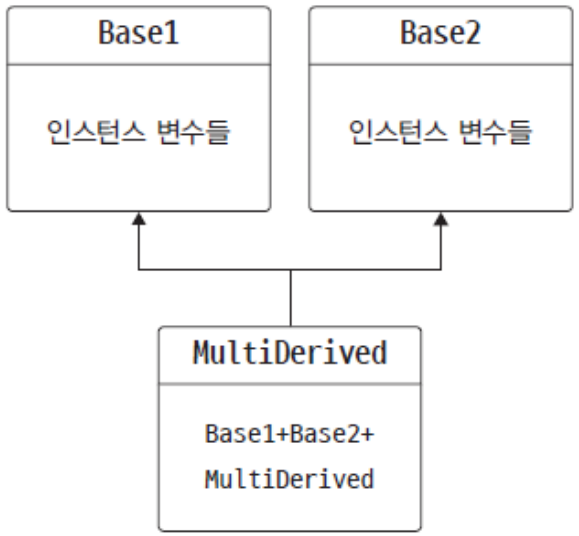

obj = Child()
print(obj.money) # 오류
```

```
...
AttributeError: 'Child' object has no attribute 'money'
```



다중 상속

- 파이썬은 다중상속을 지원한다.
- Subclass에서 부모가 한명 이상이므로 super() 보다는 부모클래스의 명칭으로 접근한다.




```
class Base1:
    pass

class Base2:
    pass

class MultiDerived(Base1, Base2):
    pass
```



다중

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def show(self):
        print(self.name, self.age)

 class Student:
    def __init__(self, id):
        self.id = id

    def getId(self):
        return self.id

class CollegeStudent(Person, Student):
    def __init__(self, name, age, id):
        Person.__init__(self, name, age)
        Student.__init__(self, id)

obj = CollegeStudent('Kim', 22, '100036')
obj.show()
print(obj.getId())
```

```
Kim 22
100036
```



다중상속 (확장 예제)

```python
class Base1:
    def __init__(self, v1 = 10):
        self.attr1 = v1

    def display1(self):
        print(f"Base1: {self.attr1}")

    def display(self):
        self.display1()

class Base2:
    def __init__(self, v2 = 20):
        self.attr2 = v2
    def display2(self):
        print(f"Base2: {self.attr2}")

    def display(self):
        self.display2()
```

```python
class C(Base1, Base2):
    def __init__(self, v1, v2, v3):
        Base1.__init__(self, v1)
        Base2.__init__(self, v2)
        self.attr3 = v3

    def print(self):
        print(f"{self.attr1}, {self.attr2}, {self.attr3}")
        self.display1()
        self.display2()
        self.display()   # Base1.display()
        Base2.display()  # Base2.display()


def test():
    c = C(v1=1, v2=2, v3=3)
    c.print()

    c2 = C(4,5,6)
    c2.print()

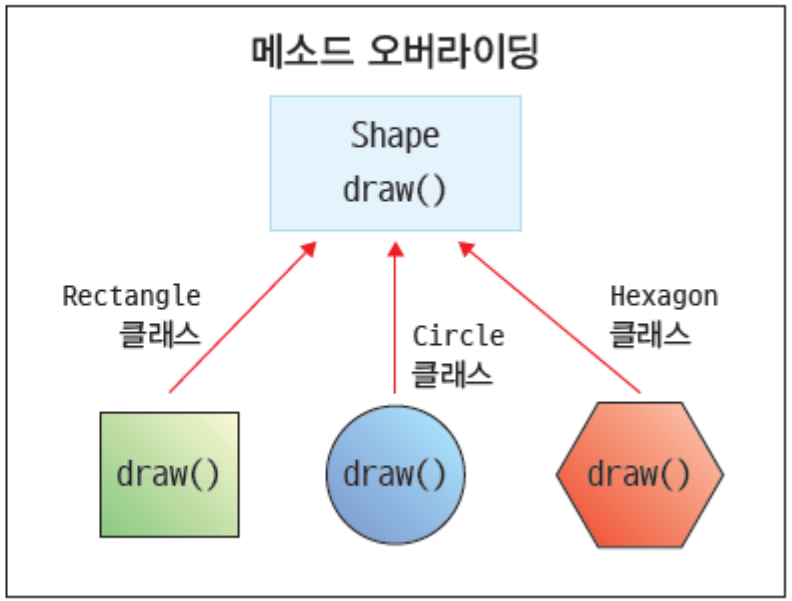
if __name__ == "__main__":
    test()
```



메소드 오버라이딩

- **"자식 클래스의 메소드가 부모 클래스의 메소드를 오버라이드(재정의)한다"**고 말한다.
- 다형성 : 함수 이름은 같으나 함수가 소속된 클래스에 따라 다른 코드가 수행되는 것 (C는 불가능, 객체지향은 가능)
- 메소드 오버라이드도 다형성의 일종임의 일종 : 수퍼클래스에서 정의된 함수를 서브클래스에서 재정의







예 제

```
import math

class Shape:
    def __init__(self):
        pass

    def draw(self):
        print("draw()가 호출됨")

    def get_area(self):
        print("get_area()가 호출됨")

class Circle(Shape):
    def __init__(self, radius=0):
        super().__init__()
        self.radius = radius

    def draw(self):
        print("원을 그립니다.")

    def get_area(self):
        return math.pi * self.radius ** 2

c = Circle(10)
c.draw()
print("원의 면적:", c.get_area())
```

```
원을 그립니다.
원의 면적: 314.1592653589793
```



예제

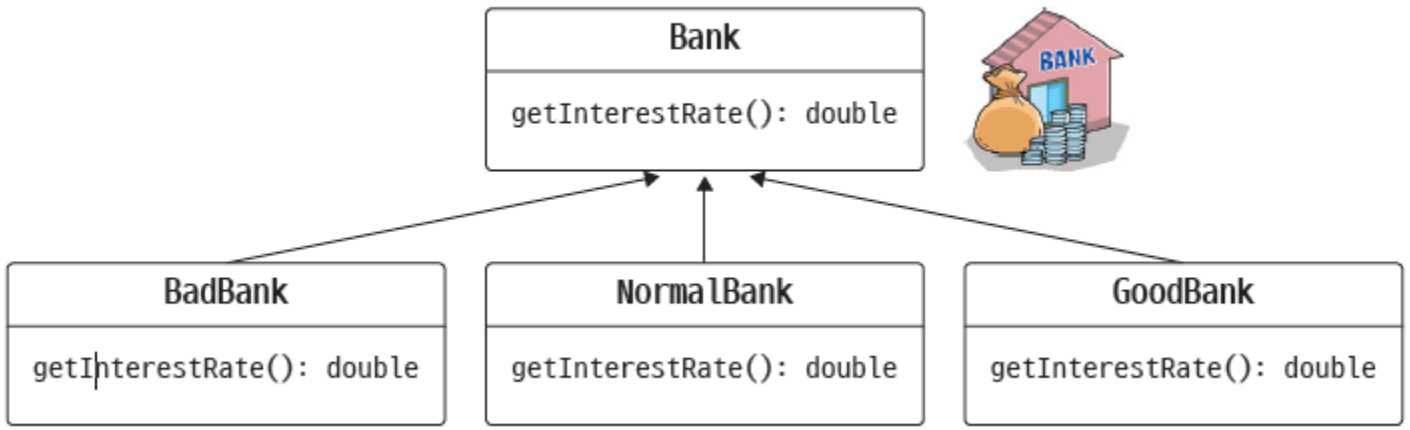
```
class Circle(Shape):
        ...
    def draw(self):
        super().draw()              # 부모 클래스의 draw()가 호출된다.
        print("원을 그립니다.")
```

```
draw()가 호출됨
원을 그립니다.
```



Lab: Bank 클래스

- 은행에서 대출을 받을 때, 은행마다 대출 이자가 다르다. 이것을 메소드 오버라이딩으로 깔금하게 해결하여 보자.




```
BadBank의 이자율: 10.0
NormalBank의 이자율: 5.0
GoodBank의 이자율: 3.0
```



Solution

```
class Bank():
        def getInterestRate(self):
                return 0.0

class BadBank(Bank):
        def getInterestRate(self):
                return 10.0;

class NormalBank(Bank):
        def getInterestRate(self):
                return 5.0;

class GoodBank(Bank):
        def getInterestRate(self):
                return 3.0;

b1 = BadBank()
b2 = NormalBank()
b3 = GoodBank()
print("BadBank의 이자율: " + str(b1.getInterestRate()))
print("NormalBank의 이자율: " + str(b2.getInterestRate()))
print("GoodBank의 이자율: " + str(b3.getInterestRate()))
```



Lab: 직원과 매니저

- 회사에 직원(Employee)과 매니저(Manager)가 있다. 직원은 월급만 있지만 매니저는 월급외에 보너스가 있다고 하자. Employee 클래스를 상속받아서 Manager 클래스를 작성한다. Employee 클래스의 getSalary()는 Manager 클래스에서 재정의된다.

```
이름: 김철수; 월급: 2000000; 보너스: 1000000
```



Solution

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def  getSalary(self):
        return salary

class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus =bonus

    def  getSalary(self):
        salary = super().getSalary()
        return salary + self.bonus

    def __repr__(self):
        return "이름: "+ self.name+ "; 월급: "+ str(self.salary)+\
               "; 보너스: "+str(self.bonus)

kim = Manager("김철수", 2000000, 1000000)
print(kim)
```



Lab: 은행 계좌

- BankAccount 클래스는 다음과 같은 인스턴스 변수와 메소드를 가진다.
  - balance –- 잔액(정수형)
  - name –- 소유자의 이름(문자열)
  - name –- 통장 번호(정수형)
  - withdraw() -- 출금 메소드
  - deposit() - 저금 메소드
- BankAccount 클래스를 상속받아서 SavingsAccount 클래스(저축예금)를 작성한다. SavingsAccount 클래스는 추가로 다음과 같은 인스턴스 변수와 메소드를 가진다.
  - interest_rate – 이자율(실수형)
  - add_interest() - 호출될 때마다 예금에 이자를 더하는 메소드
- BankAccount 클래스를 상속받아서 CheckingAccount 클래스(당좌예금)를 작성한다. CheckingAccount 클래스는 추가로 다음과 같은 인스턴스 변수와 메소드를 가진다.
  - withdraw_charge – 수표를 1회 발행할 때 수수료(정수형)
  - withdraw() - 찾을 금액에 수수료를 더해서 출금한다.



Solution

```
class BankAccount:
    def __init__(self, name, number, balance):

        self.balance = balance
        self.name = name
        self.number = number

    def withdraw(self, amount):
        self.balance -= amount
        return self.balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance
```



Solution

```
class CheckingAccount(BankAccount) :
    def __init__(self,  name, number, balance):
        super().__init__( name, number, balance)
        self.withdraw_charge = 10000        # 수표 발행 수수료

    def withdraw(self, amount):
        return BankAccount.withdraw(self, amount + self.withdraw_charge)

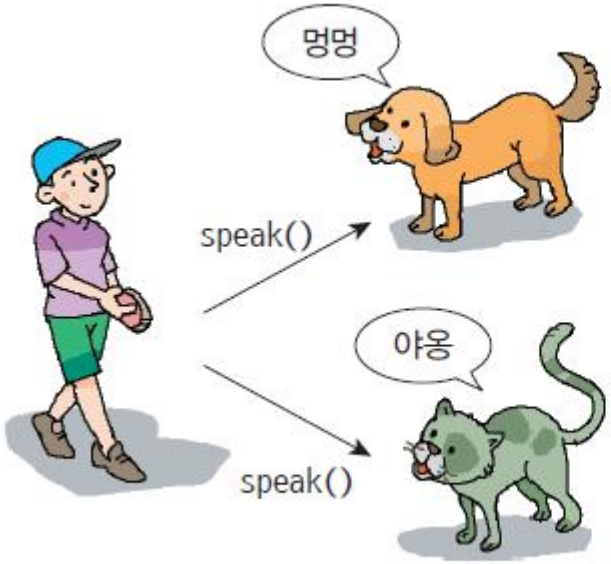
a1 = SavingsAccount("홍길동", 123456, 10000, 0.05)
a1.add_interest()
print("저축예금의 잔액=", a1.balance)

a2 = CheckingAccount("김철수", 123457, 2000000)
a2.withdraw(100000)
print("당좌예금의 잔액=", a2.balance)
```



다형성

- **다형성(polymorphism)**은 **“많은(poly)+모양(morph)“**이라는 의미로서 주로 프로그래밍 언어에서 하나의 식별자로 다양한 타입(클래스)을 처리하는 것을 의미한다.

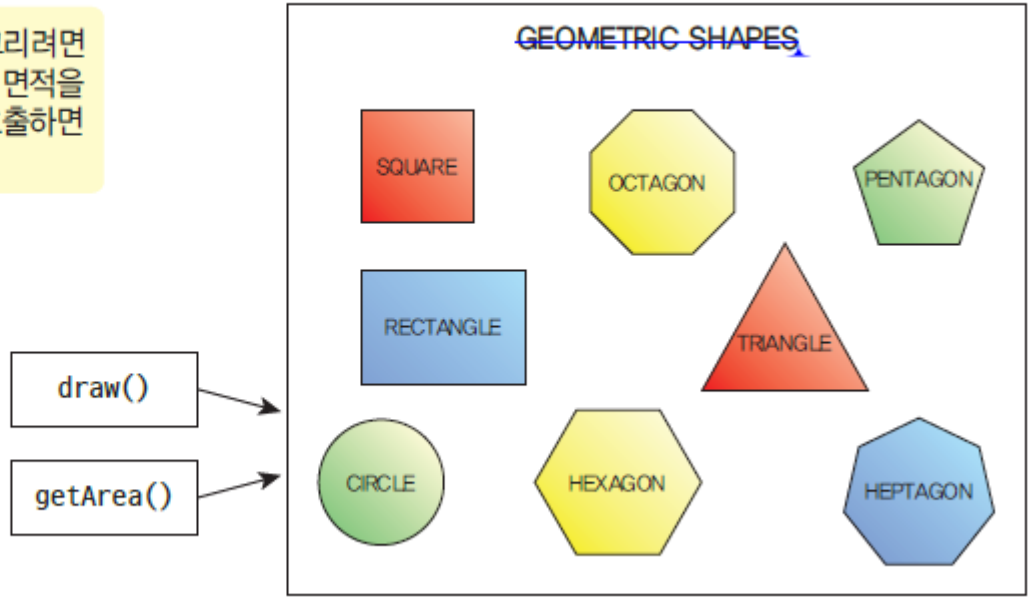






다형성의 예


도형의 타입에 상관없이 도형을 그리려면 무조건 draw()를 호출하고 도형의 면적을 계산하려면 무조건 getArea()를 호출하면 됩니다.


draw()
getArea()
GEOMETRIC SHAPES
SQUARE
OCTAGON
PENTAGON
RECTANGLE
TRIANGLE
CIRCLE
HEXAGON
HEPTAGON



상속과 다형성

```
class Shape:
    def __init__(self, name):
        self.name = name
    def getArea(self):
        raise NotImplementedError("이것은 추상메소드입니다. ")

class Circle(Shape):
    def __init__(self, name, radius):
        super().__init__(name)
        self.radius = radius

    def getArea(self):
        return  3.141592*self.radius**2

class Rectangle(Shape):
    def __init__(self, name, width, height):
        super().__init__(name)
        self.width = width
        self.height = height

    def getArea(self):
        return  self.width*self.height
```



상속과 다형성

```
shapeList = [ Circle("c1", 10), Rectangle("r1", 10, 10) ]
for s in shapeList:
        print(s.getArea())
```

```
314.1592
100
```



내장 함수와 다형성

```
mylist = [1, 2, 3]                  # 리스트
print("리스트의 길이=", len(mylist))

s = "This is a sentense"            # 문자열
print("문자열의 길이=", len(s))

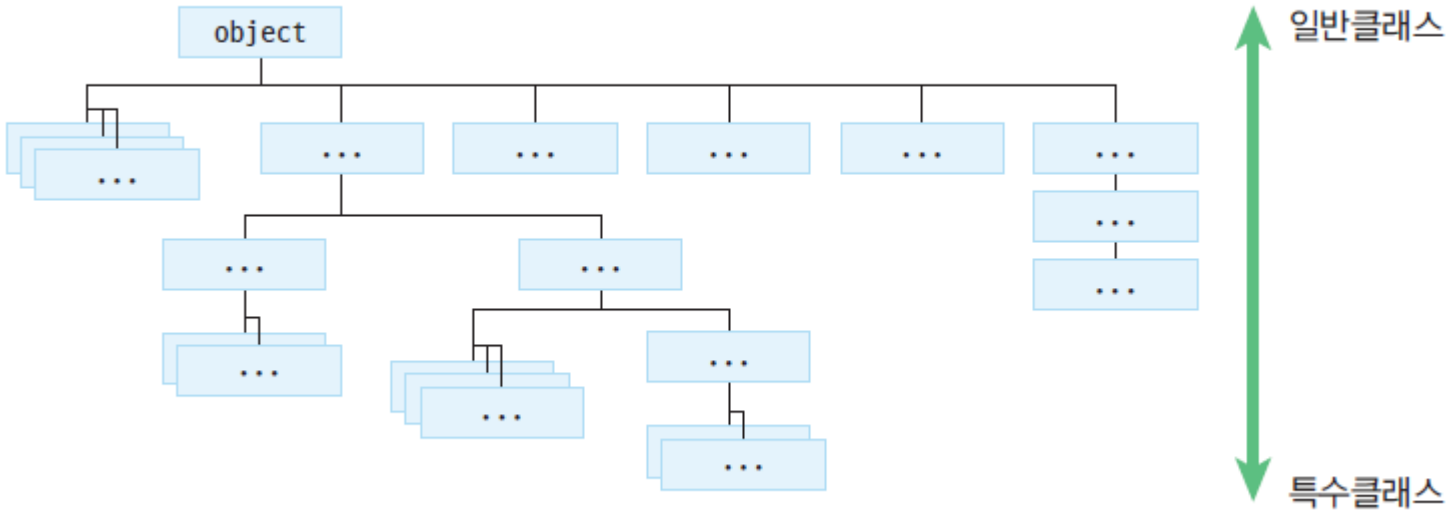
d = {'aaa': 1, 'bbb': 2}            # 딕셔너리
print("딕셔너리의 길이=", len(d))
```

```
리스트의 길이= 3
문자열의 길이= 18
딕셔너리의 길이= 2
```



object 클래스

- 모든 클래스의 맨 위에는 object 클래스가 있다고 생각하면 된다.





object 클래스의 메소드

메소드	
__init__ (self [,args...])	생성자 예 obj = className(args)
__del__(self)	소멸자 예 del obj
__repr__(self)	객체 표현 문자열 반환 예 repr(obj)
__str__(self)	문자열 표현 반환 예 str(obj)
__cmp__ (self, x)	객체 비교 예 cmp(obj, x)



__repr__() 메소드

```
class Book:
        def __init__(self, title, isbn):
                self.__title = title
                self.__isbn = isbn
        def __repr__(self):
                return "ISBN: "+ self.__isbn+ "; TITLE: "+ self.__title

book = Book("The Python Tutorial", "0123456")
print(book)
```

```
ISBN: 0123456; TITLE: The Python Tutorial
```



__str__() 메소드

```
class MyTime:
        def __init__(self, hour, minute, second=0):
                self.hour = hour
                self.minute = minute
                self.second = second
        def __str__(self):
                return '%.2d:%.2d:%.2d' % (self.hour, self.minute, self.second)

time = MyTime(10, 25)
print(time)
```

```
10:25:00
```



__repr__ 과 __str__ 차이점

구분	__repr__ (Representation)	__str__ (String)
목적	개발자가 디버깅하거나 객체의 정확한 상태를 파악하기 위함	최종 사용자(User)가 읽기 쉬운 가독성 높은 정보를 제공하기 위함
주요 사용자	개발자 (Developer-oriented)	일반 사용자 (User-oriented)
호출 시점	repr(obj), 대화형 인터페이스(REPL/Jupyter)에서 객체 출력 시, 리스트 등 컬렉션 내부 객체 출력 시	print(obj), str(obj), f"{obj}" 포맷팅 사용 시
지향점	가능하면 "동일한 객체를 다시 생성할 수 있는 실행 가능한 Python 코드 형태" 권장	"사람이 읽기 편한 형태" 권장



__repr__ 과 __str__ 차이점

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        # 객체를 다시 생성할 수 있는 코드 형태 표현
        return f"Person(name={self.name!r}, age={self.age})"

    def __str__(self):
        # 사람이 읽기 쉬운 설명 형태
        return f"{self.name} ({self.age}세)"

p = Person("홍길동", 20)

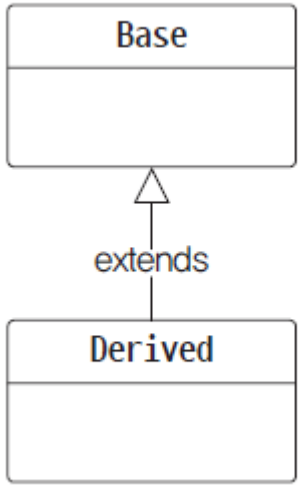
print(p)        # __str__ 호출 -> 홍길동 (20세)
print(repr(p))  # __repr__ 호출 -> Person(name='홍길동', age=20)
print([p])      # 리스트 안의 객체는 __repr__ 호출 -> [Person(name='홍길동', age=20)]
```

```
Person(name='홍길동', age=20)
[Person(name='홍길동', age=20)]
```



클래스 관계: 상속

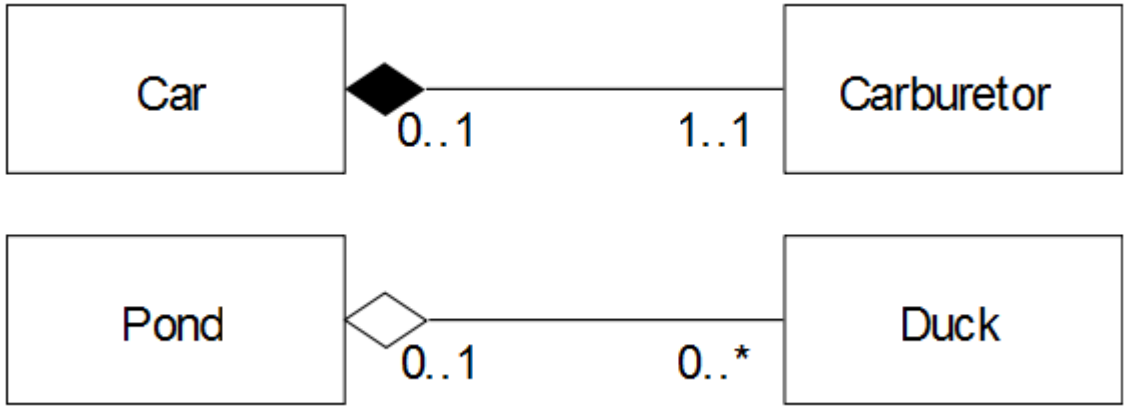
- is-a 관계: 상속
  - 승용차는 차량의 일종이다(Car is a Vehicle).
  - 강아지는 동물의 일종이다(Dog is an animal).
  - 원은 도형의 일종이다(Circle is a shape)





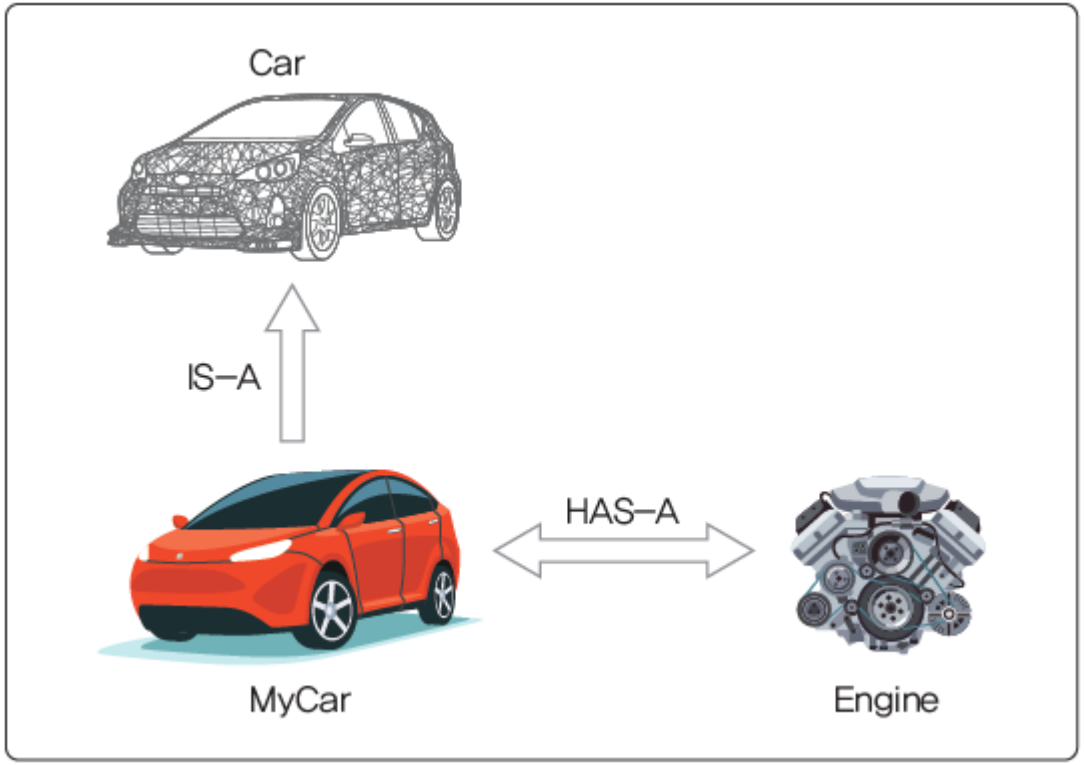
클래스 관계:구성

- has-a 관계: 구성
  - 도서관은 책을 가지고 있다(Library has a book).
  - 거실은 소파를 가지고 있다(Living room has a sofa).





Is-a vs has-a





예

```
class Animal(object):
    pass

class Dog(Animal):
    def __init__(self, name):
        self.name = name

class Person(object):
    def __init__(self, name):
        self.name = name
        self.pet = None


dog1 = Dog("dog1")
person1 = Person("홍길동")
person1.pet = dog1
```



Lab: Card와 Deck

- 카드를 나타내는 Card 클래스를 작성하고 52개의 Card 객체를 가지고 있는 Deck 클래스를 작성한다. 각 클래스의 __str__() 메소드를 구현하여서 덱 안에 들어 있는 카드를 다음과 같이 출력한다.

```
['클럽 에이스', '클럽 2', '클럽 3', '클럽 4', '클럽 5', '클럽 6', '클럽 7', '클럽 8', '클럽 9', '클럽 10', '클럽 잭', '클럽 퀸', '클럽 킹', '다이아몬드 에이스', '다이아몬드 2', '다이아몬드 3', '다이아몬드 4', '다이아몬드 5', '다이아몬드 6', '다이아몬드 7', '다이아몬드 8', '다이아몬드 9', '다이아몬드 10', '다이아몬드 잭', '다이아몬드 퀸', '다이아몬드 킹', '하트 에이스', '하트 2', '하트 3', '하트 4', '하트 5', '하트 6', '하트 7', '하트 8', '하트 9', '하트 10', '하트 잭', '하트 퀸', '하트 킹', '스페이드 에이스', '스페이드 2', '스페이드 3', '스페이드 4', '스페이드 5', '스페이드 6', '스페이드 7', '스페이드 8', '스페이드 9', '스페이드 10', '스페이드 잭', '스페이드 퀸', '스페이드 킹']
```



Solution

```
class Card:
    suitNames = ['클럽', '다이아몬드', '하트', '스페이드']
    rankNames = [None, '에이스', '2', '3', '4', '5', '6', '7',
                 '8', '9', '10', '잭', '퀸', '킹']

    def __init__(self, suit, rank):
        self.suit = suit
        self.rank = rank

    def __str__(self):
        return Card.suitNames[self.suit]+" "+\
                        Card.rankNames[self.rank]
```



Solution

```
class Deck:
    def __init__(self):
        self.cards = []

        for suit in range(4):
            for rank in range(1, 14):
                card = Card(suit, rank)
                self.cards.append(card)
    def __str__(self):
        lst = [str(card) for card in self.cards]
        return str(lst)

deck = Deck()                    # 덱 객체를 생성한다.
print(deck)                      # 덱 객체를 출력한다. __str__()이 호출된다.
```



Lab: 학생과 강사

- 일반적인 사람을 나타내는 Person 클래스를 정의한다. Person 클래스를 상속받아서 학생을 나타내는 클래스 Student와 선생님을 나타내는 클래스 Teacher를 정의한다.

```
이름=홍길동
주민번호=12345678
수강과목=['자료구조']
평점=0
이름=김철수
주민번호=123456790
강의과목=['Python']
월급=3000000
```



Solution

```
class Person:
    def __init__(self, name, number):
        self.name = name
        self.number = number

class Student(Person):
    UNDERGRADUATE=0
    POSTGRADUATE = 1

    def __init__(self, name, number, studentType ):
        super().__init__(name, number)
        self.studentType = studentType
        self.gpa=0
        self.classes = []

    def enrollCourse(self, course):
        self.classes.append(course)

    def __str__(self):
        return  "\n이름="+self.name+ "\n주민번호="+self.number+\
            "\n수강과목="+ str(self.classes)+ "\n평점="+str(self.gpa)
```



Solution

```
class Teacher(Person):
    def __init__(self, name, number):
        super().__init__(name, number)
        self.courses = []
        self.salary=3000000

    def assignTeaching(self, course):
        self.courses.append(course)

    def __str__(self):
        return  "\n이름="+self.name+ "\n주민번호="+self.number+\
            "\n강의과목="+str(self.courses)+ "\n월급="+str(self.salary)

hong = Student("홍길동", "12345678", Student.UNDERGRADUATE )
hong.enrollCourse("자료구조")
print(hong)

kim  = Teacher("김철수", "123456790")
kim.assignTeaching("Python")
print(kim)
```



Lab: 학생과 강사

- 은행 계좌를 나타내는 클래스 BankAccount 클래스를 정의한다.

```
저축예금의 잔액= 10500.0
당좌예금의 잔액= 1890000
```



Solution

```
class BankAccount:
    def __init__(self, name, number, balance):

        self.balance = balance
        self.name = name
        self.number = number


    def withdraw(self, amount):
        self.balance -= amount
        return self.balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance
```



Solution

```
class SavingsAccount(BankAccount) :
    def __init__(self,  name, number, balance,  interest_rate):

        super().__init__( name, number, balance)
        self.interest_rate =interest_rate

    def set_interest_rate(self, interest_rate):
        self.interest_rate = interest_rate

    def get_interest_rate(self):
        return self.interest_rate

    def add_interest(self):                    # 예금에 이자를 더한다.
        self.balance += self.balance*self.interest_rate
```



Solution

```
class CheckingAccount(BankAccount) :
    def __init__(self,  name, number, balance):
        super().__init__( name, number, balance)
        self.withdraw_charge = 10000        # 수표 발행 수수료

    def withdraw(self, amount):
        return BankAccount.withdraw(self, amount + self.withdraw_charge)

a1 = SavingsAccount("홍길동", 123456, 10000, 0.05)
a1.add_interest()
print("저축예금의 잔액=", a1.balance)

a2 = CheckingAccount("김철수", 123457, 2000000)
a2.withdraw(100000)
print("당좌예금의 잔액=", a2.balance)
```



Lab: Vehicle와 Car, Truck

- 일반적인 운송수단을 나타내는 Vehicle 클래스를 상속받아서 Car 클래스와 Truck 클래스를 작성해보자.

```
truck1: 트럭을 운전합니다.
truck2: 트럭을 운전합니다.
car1: 승용차를 운전합니다.
```



Solution

```
class Vehicle:
    def __init__(self, name):
        self.name = name

    def drive(self):
        raise NotImplementedError("이것은 추상메소드입니다. ")

    def stop(self):
        raise NotImplementedError("이것은 추상메소드입니다. ")
```



Solution

```
class Car(Vehicle):
    def drive(self):
        return '승용자를 운전합니다. '

    def stop(self):
        return '승용자를 정지합니다. '


class Truck(Vehicle):
    def drive(self):
        return '트럭을 운전합니다. '

    def stop(self):
        return '트럭을 정지합니다. '

cars = [Truck('truck1'), Truck('truck2'),  Car('car1')]

for car in cars:
    print( car.name + ': ' + car.drive())
```



가시성 (Visibility)

- 파이썬은 C++/자바와 달리 접근 제어자(public, protected, private)가 엄격하게 적용되지 않습니다. 하지만 관례적으로 다음과 같이 가시성을 표현
  - public: 이름 앞에 밑줄 없음 (예: value)
    → 어디서나 접근 가능
  - protected: 이름 앞에 밑줄 1개 (예: _value)
    → 하위 클래스에서 접근 권장, 외부에서 직접 접근은 권장하지 않음
  - private: 이름 앞에 밑줄 2개 (예: __value)
    → 클래스 내부에서만 접근, 이름이 맹글링(name mangling)되어 외부/하위 클래스에서 직접 접근 어렵게 만듦



가시성 예제

```
class Base:
    def __init__(self):
        self.public_value = 1           # public
        self._protected_value = 2       # protected
        self.__private_value = 3        # private


class Child(Base):
    def show(self):
        print(self.public_value)        # 가능
        print(self._protected_value)    # 가능
        # print(self.__private_value)   # 불가 (AttributeError)


class Other:
    def show(self, obj: Base):
        print(obj.public_value)         # 가능
        print(obj._protected_value)     # 가능 (하지만 권장되지 않음)
        # print(obj.__private_value)    # 불가 (AttributeError)
```



추상 함수 (abstract methods)

- 파이썬에서 추상 메서드(abstract method)
  - abc 모듈의 ABC(Abstract Base Class)와 @abstractmethod 데코레이터 (Java 의 annotation)를 사용해 정의
  - 추상 메서드는 반드시 하위 클래스에서 구현해야 하며, 추상 메서드를 하나라도 가진 클래스는 Abstract class로서 이 클래스는 직접 인스턴스를 만들 수 없음!



추상함수 예제

- 추상클래스는 abc.ABC 클래스로부터 상속받아야 함
- 추상메소드는 @abstractmethod 데코레이터(장식함수)를 선언하고 정의해야 함
  - Sound() 함수는 추상함수로 pass (아무것도 하지 않음)
  - 자식클래스는 반드시 추상함수를 재정의해야 함

```
from abc import ABC, abstractmethod


class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass


class Dog(Animal):
    def sound(self):
        return "멍멍"

a = Animal()  # 에러: 추상 클래스는 인스턴스화 불가
d = Dog()
print(d.sound())  # "멍멍"
```



정적 멤버 (static variable, static methods)

- 파이썬에서 정적 메서드
  - 파이썬에서 static member(정적 멤버, 클래스 변수)는 클래스 전체에서 공유되는 변수
  - 클래스 변수, 클래스 메소드라고 함
  - 메서드의 경우 @staticmethod 데코레이터를 사용해 정적 메서드를 정의
  - 정적 변수는 클래스 안에 self 가 아닌 일반 변수로 선언하고 클래스.변수 형태로 접근



정적 멤버 예제

```
class Counter:
    count = 0  # 클래스 변수(정적 멤버)


    def __init__(self):
        self.__object_counter = 1   # public

    @staticmethod
    def increment_class():
        Counter.count += 1

    def increment_object(self):
        self.__object_counter += 1  # private

if __name__ == "__main__":
     Counter.increment_class()
     print(Counter.count)                  # 1

     c = Counter()                         # 인스턴스 생성
     print(c.object_counter)    # 1
     c.increment_object()
     print(c.__object_counter)  # private 접근 불가
```



연습문제 (추상클래스 다루기)

- 동물(Animal)을 울음을 추상 메소드로 하는 추상클래스로 정의하고,
  - 개(Dog)와 고양이(Cat)를 구체적인 클래스로 정의한 후,
  - 각 동물의 울음소리를 출력하는 프로그램을 작성하시오.
  - 각 동물객체는 자신의 이름을 속성으로 가지며,
  - 생성자에서 이름을 초기화합니다.
  - 추가적으로 Animal 클래스는 현재 생성된 동물 객체의 수를 클래스 변수로 가지고,
  - 정적 메소드로 현재 동물 객체의 수를 반환하는 메소드를 구현
  - 객체가 생성(__init__)될 때 객체의 수를 증가하고 삭제(__del__(self)) 될 때 객체의 수를 감소

- 테스트
  - 개나 고양이 객체들을 리스트로 생성하고 개나 Animal 변수로 받아 동적 바인딩이 되는지 확인하시오.
  - 모든 객체들이 울게 하는 함수를 작성하여 테스트하시오.



테스트 코드

```
if __name__ == "__main__":

    dog : Dog = Dog("바둑이")
    print(dog.sound())  # Output: 멍멍

    cat = Cat("야옹이")
    print(cat.sound())  # Output: 야옹

    any : Animal = Dog("바둑이2")
    print(any.sound())  # Output: 멍멍
    any : Animal = Cat("야옹이2")
    print(any.sound())  # Output: 야옹

    alist : list[Animal] = [Dog("바둑이a"), Cat("나비b"), Dog("멍멍이c"), Cat("야옹이d")]
    print(alist)

    animal_sound(alist[2])
    animal_sounds(alist)

    print(f"총 동물의 수: {Animal.get_animal_count()}") # 8

    # 올바른 객체 삭제 방법
    del dog  # del 키워드 사용
    del cat  # del 키워드 사용

    print(f"총 동물의 수: {Animal.get_animal_count()}") # 6
```



Dunder methods

- Dunder
  - Double Underlines 의 약자
  - 파이썬에서는 Underline은 특별한 의미로 사용할 때가 많음
  - __XXX__ 메소드는 자동으로 호출되는 경우가 많음
  - 이러한 메소드를 재정의 하여 사용자 코드를 수행하도록 할 수 있음
- Dunder Methods 예 : dir(object)
  - __init__(self,…) : 클래스 생성함수, 객체 만들어질 때 자동 실행 ➔ a = A()
  - __str__(self) : 클래스 객체를 문자열로 형병변환 할 때 자동 실행 ➔ print(a)
  - __eq__(self,other) : 이 객체와 다른 객체를 같은지 여부 ➔ a == b
  - __gt__(self,other) : 이 객체가 다른 객체보다 큰지 여부 ➔ a > b

```
>>> dir(object)
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
'__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__']
```



Dunder methods : 예제

```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"Student(name={self.name}, age={self.age})"

    def __eq__(self, other):
        if isinstance(other, Student):
            return self.name == other.name and self.age == other.age
        return False

    def __del__(self):
        print(f"Student {self.name} is being deleted")


if __name__ == "__main__":
    student1 = Student("Alice", 20)
    student2 = Student("Bob", 22)
    student3 = Student("Alice", 20)

    print(student1)  # Calls __str__
    print(student2)  # Calls __str__

    print(student1 == student2)  # Calls __eq__
    print(student1 == student3)  # Calls __eq__

    del student1  # Calls __del__
```



파이썬 데코레이터

- 파이썬 데코레이터(decorator)는 함수나 클래스의 동작을 수정하거나 확장할 때 사용하는 문법
  - 함수(또는 메서드) 정의 위에 @데코레이터이름 형태로 사용
  - 해당 함수가 호출될 때 데코레이터 함수 내에서 해당 함수가 실행
  - 데코레이터 함수의 정의는 다음 형식을 따라 wrapper 이름의 멤버 메소드를 정의
    - def *deco_func* (func) :

```
        def wrapper(*args, **kwargs) :
            ...
            result = func(*args, **kwargs)
            …
            return result
        return wrapper
```



파이썬 데코레이터(annotation) 예제

```
def simple_decorator(func):
    def wrapper():
        print("함수 실행 전") # 추가적인 코드
        result = func()   # 목적 함수 실행
        print( " 함수 실행 후 " )  # 추가적인 코드
        return result
    return wrapper

@simple_decorator
Def say_hello():
    print( " Hello, World! " )

If __name__ ==  " __main__ " :
    print( " === 예제 1: 기본 데코레이터 === " )
    say_hello()

# 함수 실행 전
# Hello, World!
# 함수 실행 후
```



기존 함수를 확장(데코레이터를 통한)하기 위한 방법 2

- 이미 작성된 함수의 기능을 확장할 때 2가지 방법
  - 1. 함수 재할당
    - 장식이 된 함수명을 기존 함수명으로 그대로 사용하게 하고 싶을 때 : 기존 클라이언트 코드 변경 없이
    - 원본 함수명 = decorator(원본 함수명)
  - 2. 다른 함수명
    - 장식이 된 함수명을 새로운 함수명으로 명명하여 사용해도 좋을 때 : 새롭게 클라이언트 함수를 짤 때
    - 신규 함수명 = decorator(원본 함수명)



파이썬 데코레이터(annotation) 예제 2

```
def say_hello():
    print("Hello, World!")

def simple_decorator(func):
    def wrapper():
        print("함수 실행 전")
        result = func()
        print("함수 실행 후")
        return result
    return wrapper

original_hello = say_hello                                          # 원본 백업
say_hello = simple_decorator(original_say_hello)                # 1. 함수재할당
decorated_say_hello = simple_decorator(original_say_hello)      # 2. 다른 함수명이름

if __name__ == "__main__":
    say_hello()
    decorated_say_hello()

# 함수 실행 전
# Hello, World!
# 함수 실행 후
```



이번 장에서 배운 것

- 상속은 다른 클래스를 재사용하는 탁월한 방법이다. 객체와 객체간의 is-a 관계가 성립된다면 상속을 이용하도록 하자.
- 상속을 사용하면 중복된 코드를 줄일 수 있다. 공통적인 코드는 부모 클래스를 작성하여 한 곳으로 모으도록 하자.
- 상속에서는 부모 클래스의 메소드를 자식 클래스가 재정의할 수 있다. 이것을 메소드 오버라이딩이라고 한다.





Q & A



